Bada Source-Code Generation for an Autonomous Electric Car

A Gamma-Manifold Entropy Equation and the Jones Polynomial of a Thermal-Energy Body for Policy Synthesis and Audio-Visual Rendering

Masaaki Yamaguchi （山口 雅旭） · Global Differential Manifold Research · package `omega_bluebird_ev_pkg` · 2026

> ⚠️ **Scope / Honesty Note.** This is a **conceptual / simulation** report. The entropy, manifold and Jones mathematics is genuine (Shannon entropy, Lanczos Γ, Kauffman bracket). The "source-code generation from an entropy invariant" is a small **deterministic program synthesizer** — a generative grammar seeded by an invariant — **not** a trained large language model. The generated driving policy is illustrative and **must not be deployed in a real vehicle**: it has no sensor stack, no validation, and no safety case.

要旨 / Abstract

本稿は、Bluebird 車体の自動走行電気自動車のオペレーティングシステムとしての生成AIを、Bada 言語で記述し、ガンマ関数の大域的部分積分多様体のエントロピー方程式で Bada ソースコードを生成し、熱エネルギー体の Jones 多項式で周囲の映像化と音の映像化を行う実装 `omega_bluebird_ev_pkg` を報告する。

We present a generative-AI operating system for a Bluebird-styled electric car. Its headline capability is a **source-code generator**: the entropy of the gamma-function global partial-integral manifold is measured over a percept stream and used to synthesize a Bada autonomous-driving policy, while the Jones polynomial of a thermal-energy body drives an environment **image** (PPM) and an engine **sound** (WAV). We define the entropy equation $\Xi = \beta(H{+}1, M{+}1)/\log(N{+}2)$, show the synthesizer is deterministic and context-sensitive, and report measured invariants and generated parameters.

---

1. Introduction

Program synthesis conditioned on a scalar invariant is a compact model of "generative" behaviour: identical context regenerates identical code, and a changed context yields a different but valid program. We apply this to an autonomous-driving "operating system" written in the Bada language, seeding the synthesis with an entropy functional on the project's global partial-integral manifold, and rendering the surroundings and engine note from a knot-

theoretic thermal invariant. The physical driving here is a toy; the contribution is the honest, reproducible pipeline from an invariant to source code, image and sound.

2. The gamma-manifold entropy equation

Let a percept file define a byte measure $p_1 \geq p_2 \geq \cdots \geq p_m$ with m distinct symbols over N bytes. Define the Shannon entropy and the global partial-integral of the manifold element,

$$H = -\sum_i p_i \log_2 p_i, \qquad d\mu(x) = \frac{1}{(x \log x)^2}, \qquad M = \sum_{i=1}^{m} p_i \frac{1}{(x_i \log x_i)^2}, \quad x_i = i + 1.$$

The entropy equation output is the zeta-gauge scalar

$$\Xi = \frac{\beta(H+1, M+1)}{\log(N+2)}, \qquad \beta(a,b) = \frac{\Gamma(a)\Gamma(b)}{\Gamma(a+b)},$$

with Γ evaluated by the Lanczos approximation. Ξ is the seed for all downstream generation.

3. Jones polynomial of the thermal-energy body

The battery/inverter heat flow is encoded as a knot diagram D; its Kauffman bracket and temperature-weighted thermal invariant are

$$\langle D \rangle = \sum_s A^{a(s)-b(s)} d^{|s|-1}, \quad d = -A^2 - A^{-2}, \qquad J_{\text{th}} = \langle D \rangle \, \beta\left(1 + \tfrac{T}{300}, 1 + \tfrac{300}{T}\right).$$

4. Bada source-code generation

The generator seeds an `xorshift` stream from the mantissa/exponent of Ξ and J_{th} and emits a Bada policy in which every literal is a function of the invariants:

$$v_{\text{cap}} = 8 + \text{rnd}[0, 14] + 6\Xi, \qquad d_{\text{brake}} = 6 + \text{rnd}[0, 10] + 20M,$$

$$k_{\text{steer}} = 0.15 + 0.25(H/8), \qquad \text{caution} = \begin{cases} 1.0 & J_{\text{th}} < 0 \\ 0.6 & \text{otherwise,} \end{cases}$$

and the order of the two safety rules is chosen by $\Xi \bmod 2$. Because the seed is a pure function of the context, the map *context* → *source* is deterministic; different contexts produce different valid programs. A representative fragment (`bada/generated_policy.bada`):

```
class BluebirdAutopilot <- TupleSpace {
  set v_cap   = 16.236   # from Xi
  set brake_d = 12.626   # from M
  set steer_k = 0.301    # from H
  set caution = 1.00     # from sign of Jones(thermal)
  operator -< (scene) {
    if scene.obstacle_dist < brake_d * caution { return DRV_BRAKE; }
    if abs(scene.lane_offset) > 0.20 {
      return scene.lane_offset > 0 ? DRV_STEER_LEFT : DRV_STEER_RIGHT; }
    if scene.speed < v_cap { return DRV_ACCEL; }
    return DRV_HOLD_LANE;
  }
}
```

5. Environment and sound visualization

周りの映像化 (image): a top-down scene is rasterized to a binary PPM; the manifold entropy Ξ and H modulate road texture and lane-marking cadence.

音の映像化 (sound): a 16-bit PCM WAV is synthesized by additive synthesis whose fundamental and timbre come from the thermal invariants,

$$f_0 = 110 + (\lfloor |\langle D \rangle| \cdot 37 \bmod 550) \text{ Hz}, \qquad \text{spread} = 1 + \tfrac{1}{2}\lfloor J_{\text{th}} \rfloor,$$

with an ASCII spectrogram printed alongside. For the trefoil at $A = 1.2$, $T = 320$ K: $f_0 = 368$ Hz, tremolo 8.8 Hz.

6. The autonomous-driving kernel

A π-softmax policy over {hold, steer-left, steer-right, brake, accel} scores actions from lane offset, obstacle distance and a braking envelope $d_{\text{env}} = 8 + 0.6v$, selecting argmax and pushing telemetry to the Akashic TupleSpace.

7. Results

Program output for `examples/road_scene.txt`:

Quantity	Value
Shannon entropy H	4.8400 bits
Manifold integral M	0.09212
Symbol extent m	60
Entropy invariant Ξ	0.03338
Thermal $\langle D \rangle$ / J_{th}	-6.9737 / -1.1632
Generated v_{cap} / d_{brake}	16.236 / 12.626
Env image / sound	320×180 PPM / 44.1 kHz WAV

Regenerating from the same file yields byte-identical source (**determinism**); a lower-entropy percept file produces different literals (**context sensitivity**).

8. Scope and limitations

The entropy equation (§2), the Kauffman bracket, and the PPM/WAV renderers are genuine. The synthesizer is a seeded generative grammar, not a learned model. The driving policy is a toy with no perception, validation or safety case and **must not be used in a real car.**

9. Conclusion

We demonstrated a reproducible pipeline that turns a gamma-manifold entropy invariant into Bada source code, and a thermal knot invariant into image and sound, all under one Bada-language operating system. The construction is transparent and deterministic, which makes the "generative" behaviour easy to audit — and easy to keep honestly separated from any physical claim.

References

1. C. E. Shannon, *A mathematical theory of communication*, Bell Syst. Tech. J. **27** (1948) 379-423.
2. C. Lanczos, *A precision approximation of the gamma function*, SIAM J. Numer. Anal. **1** (1964) 86-96.
3. L. H. Kauffman, *State models and the Jones polynomial*, Topology **26** (1987) 395-407.
4. M. Yamaguchi, *Bada Language, BadaOS and the TupleSpace framework*, Global Differential Manifold Research, 2025.